

PROGRAMMING FUNDAMENTALS

Spring 2024

1st Feb

Shazia Rizwan

- Email: shazia_rizwan@bnu.edu.pk
- Office hours: Mon -Fri 10:00am - 4:00pm
(I would appreciate if you inform before coming over for extra time)

Google Classroom Code

ky653pp

General Rules

- You may expect a *Marked* activity in any class (unannounced)
- No Makeup for an announced marked activity
- Makeup allowed for a take home assignment with penalty
- Attendance once marked will not be changed
- Zero tolerance towards cheating and plagiarism

General Rules (Contd.)

- No marked activity will be dropped
- Keep a track of your attendance yourself - 75% required
- Feel free to see me in my office for extra help

Books - for reference

- C++ Programming: From Problem Analysis to Program Design, 8th Ed. D.S. Malik
- Starting out with C++, 9th Ed. Tony Gaddis

Books have been posted in google classroom



BELIEVE IN YOURSELF!!

Division of Marks

Class Tests	15
Assignments	20
Mid Term Test	25
Final Exam	40
Total	<u>100</u>

Let's start the course ...

- Comment on any one of the following:
- Programming is about THINKING!
- A code **not** based on logic is.....NO CODE at all
- “Any fool can write code that a computer can understand. Good programmers write code that humans can understand.”
 - **Martin Fowler**

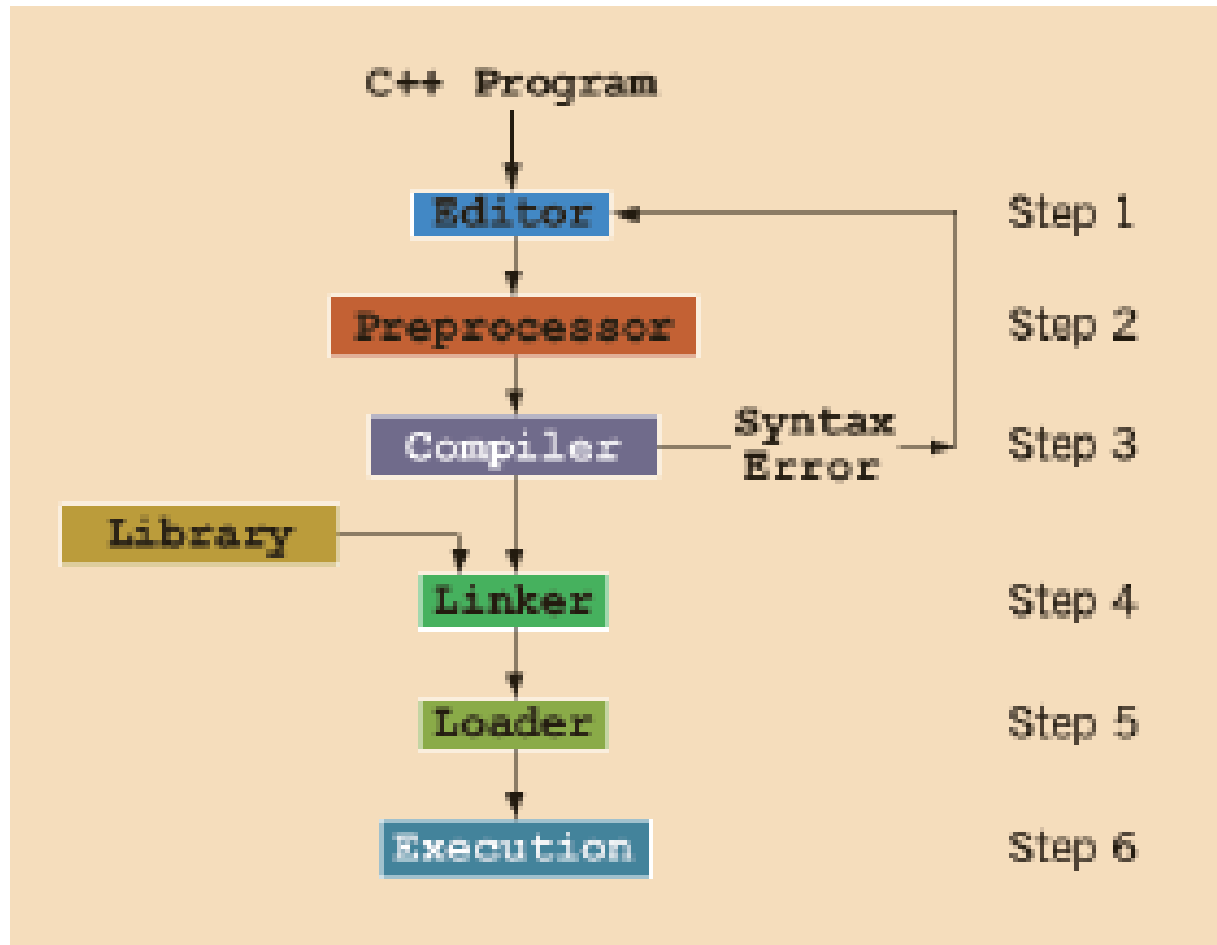
C++ Program- Lets revise

- Statements that begin with the symbol # are called preprocessor directives
- These statements are processed by a program called **preprocessor**
- After this compiler checks for syntax errors in the source code → Object code in machine language
- IDE : contains many useful programs for creating C++ programs
 - Program to display the result of the program
 - Functions are Included in library
 - Linker combines the Obj with other programs in the library

Running Object Code

- Loader → Loads the executable program into main memory
- Run the program

Processing a C++ program



Source Code

- The statements written by the programmer are called *source code*
- File they are saved in is called the *source file*

Object Code

- After the source code is saved to a file
- Need to translate it to machine language
- Translation:
 - **first phase** → a program called the *preprocessor* reads the source code
 - The preprocessor searches for special lines that begin with the `#` symbol
 - These lines contain commands that cause the preprocessor to modify the source code in some way

Source Code

- The statements written by the programmer are called *source code*
- File they are saved in is called the *source file*

Object Code

- After the source code is saved to a file
- Need to translate it to machine language
- Translation:
 - **first phase** → a program called the *preprocessor* reads the source code
 - The preprocessor searches for special lines that begin with the `#` symbol
 - These lines contain commands that cause the preprocessor to modify the source code in some way

What Is a Program Made of?

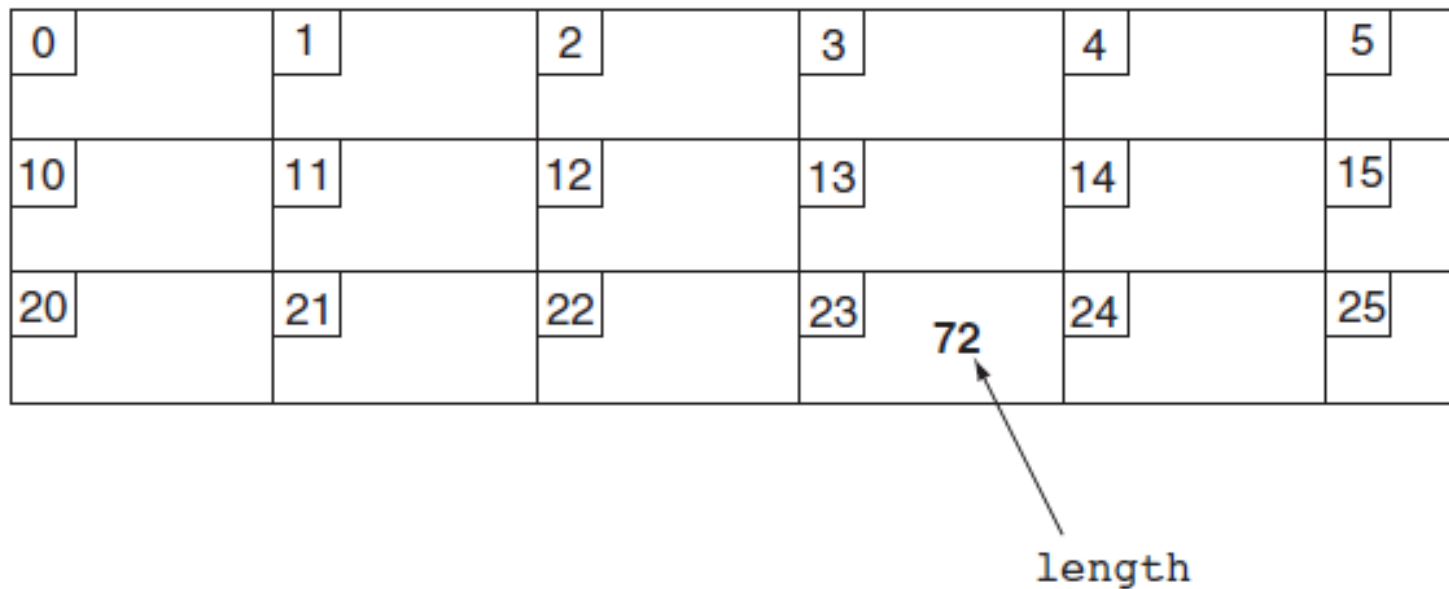
- Key words
- Programmer Defined Identifiers
- Operators
- Punctuation
- Syntax

Variables

- To store information
- To label data with a descriptive name
- For better understanding of the programmer and the reader
- A variable → a container that holds information
- The information stored in variables may change while the program is running

Memory Picture

- Variables are symbolic names that represent locations in the computer's random-access memory (RAM)



Arithmetic Operators

Operator	Meaning	Type
+	Addition	Binary
-	Subtraction	Binary
*	Multiplication	Binary
/	Division	Binary
%	Modulus	Binary

Special characters in C++

// **Double slash** → Marks the beginning of a comment

Pound sign → (hash sign) Marks the beginning of a preprocessor directive

#include <iostream>

<> **Angle brackets** → Encloses a filename when used with the #include directive

() **Parentheses** → Used in naming a function, as in int main()

Special characters in C++

{ } **Braces** → Encloses a group of statements, such as the contents of a function

“ “ **quotation marks** → Encloses a string of characters, such as a message that is to be printed on the screen

; **semicolon** → Marks the end of a complete programming statement

Rules for Variable Names

- All variable names must be one word → any sequence of characters of text and can consist of one or more characters
- Underscores are allowed and hyphens may be allowed, but **spaces are never allowed**
- **Miles_traveled** is fine but **Miles traveled** is not correct
- **Cost-per-gallon** is fine but **cost per gallon** is not

Rules for Variable Names

- Variable names can be long (like more than 200chars)
- Avoid too long or too complicated names
- Variable names can also be short; a single character is an acceptable variable name
- At times one letter rarely indicates what the variable represents, it is best to reserve this type of variable name for specific circumstances
- A, x, or N are all acceptable variable names

Rules for Variable Names

- Digits (0 to 9) are not valid variable names
- Special characters such as # or \$ are not valid characters to be included in variable names
- Many programmers use uppercase letters to distinguish one word from another within a single name. This is sometimes referred to as **CamelBack** (or CamelCase) notation
- **MilesTraveled** works as well as Miles_Traveled and may be easier to type
- **CostPerGallon** works as well as Cost_Per_Gallon and may be easier to type

Rules for Variable Names

- Most languages allow numbers to be part of variable names, but variable names may **never start with a number**
- **TaxRate_1** is fine but **1_TaxRate** is not
- **Destination2** is fine but **2Destination** is not

Assignment Statement

```
ItemPrice = 99;
```

Assigns the value 99 to the variable ItemPrice

```
NoOfItems = 10;
```

```
Bill = ItemPrice * NoOfItems;
```

Multiplies ItemPrice by NoOfItems and then assigns the resulting value of the expression on the **RIGHT** of the equals sign to the variable (Bill) on the **LEFT** side

Assigning & Reassigning

NumberX = 45;

NumberX = 97;

- NumberX first contains the value of 45
- Then value of 45 is replaced by 97
- NumberX will keep the value of 97 until some other statement reassigns it a new value
- In terms of storage locations, the value currently stored in location NumberX (45) is erased and the new value (97) is stored in its place

Assigning & Reassigning

Counter = Counter + 1;

- First the right side is evaluated, and 1 is added to the current value of the variable Counter
- Then this new value is assigned to the variable on the left

OR

- Add one to the previous value of Counter
- If the value was 15 prior to execution of this statement, its value would be 16 afterward

Writing Expressions in Programs

$$C = \frac{5(F - 32)}{9}$$

In a programming language, the formula is written as:

```
C = 5 * (F - 32) / 9
```

C is the variable for number of degrees in Celsius, and **F** is the variable for the number of degrees in Fahrenheit

Operator Precedence

Operator(s)	Operation(s)	Order of evaluation (precedence)
()	Parentheses	Evaluated first. If the parentheses are nested, the expression in the innermost pair is evaluated first. If there are several pairs of parentheses “on the same level” (i.e., not nested), they are evaluated left to right.
*, /, or %	Multiplication Division Modulus	Evaluated second. If there are several, they are evaluated left to right.
+ or -	Addition Subtraction	Evaluated last. If there are several, they are evaluated left to right.

Refer to Appendix C in your book to understand this better

Arithmetic Operator Precedence

Operator	Operation	Precedence number
()	override normal precedence rules	1
–	negation (reverses the sign of a number)	2
*, /, %	multiplication, division, and modulus arithmetic	3
+, –	addition and subtraction	4

Your first file

[*] test.cpp

```
1  #include <iostream>
2  using namespace std;
3  int main()
4  {
5      cout<<"Hello World";
6      return 0;
7  }
8
9  |
```

#include <iostream>

- The word include with '#' tells the compiler to include the file `iostream` into the program
- File `iostream` is a header file needed for input/ output requirements of the program
- It is called a `preprocessor directive`
- The preprocessor reads your program before it is compiled and only executes those lines beginning with a # symbol
- The preprocessor is a program that “sets up” your source code for the compiler

using namespace std;

- All the elements of the standard C++ library are declared within **std**
- Programs usually contain several items with unique names, variables
- C++ uses *namespaces* to organize the names of program entities
- The statement **using namespace std;** declares that the program will be accessing entities whose names are part of the namespace called **std**

int main ()

- When program is loaded in the memory, the control is handed over to function main () and it is the first function to be executed

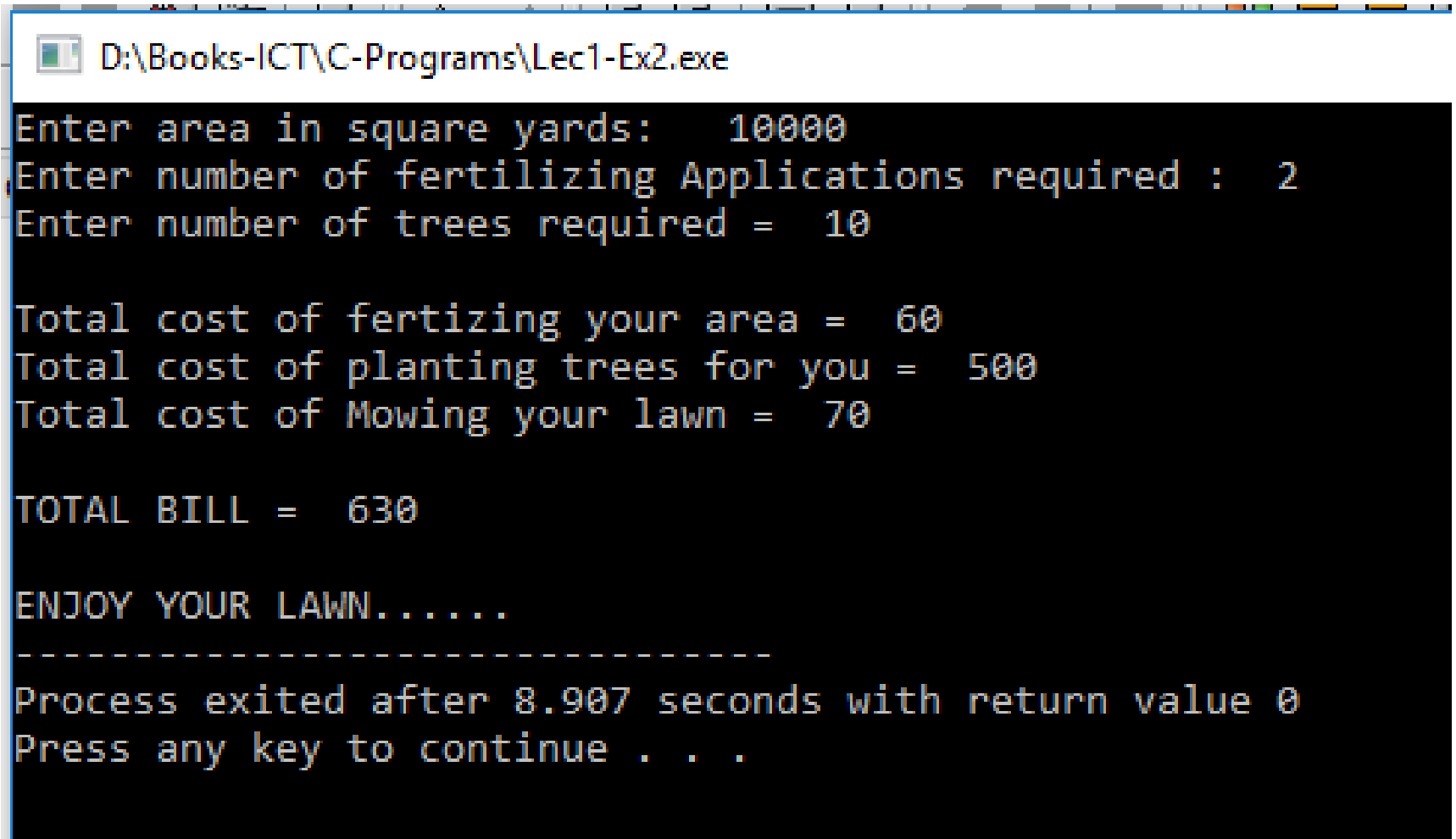
return 0;

- When a program finishes running, it sends a value to the operating system
- This returns the value of 0 to the operating system, which means that the program ended successfully

Exercise-1

- Tom and Jerry opened a new lawn service. They provide three types of services: mowing, fertilizing, and planting trees.
- The cost of mowing is \$35.00 per 5,000 square yards, fertilizing is \$30.00 per application, and planting a tree is \$50.00.
- Write a program that asks the user to enter the area of the lawn in sq yards, the number of fertilizing applications, and the number of trees to be planted.
- Display the total bill amount. (Assume that the user orders all three services.)

Sample Run of Ex-2



```
D:\Books-ICT\C-Programs\Lec1-Ex2.exe
Enter area in square yards: 10000
Enter number of fertilizing Applications required : 2
Enter number of trees required = 10

Total cost of fertizing your area = 60
Total cost of planting trees for you = 500
Total cost of Mowing your lawn = 70

TOTAL BILL = 630

ENJOY YOUR LAWN.....
-----
Process exited after 8.907 seconds with return value 0
Press any key to continue . . .
```

Chapter Reference

- D.S Malik
- Chapter - 2